

TinyLinux-IoT-KernelLab: 从零实现一个可扩展 Linux 智能终端系统

虚拟机ubuntu 16.04 开发板：正点原子阿尔法imx6ull开发板 参考教程：正点原子【正点原子】I.MX6U嵌入式Linux驱动开发指南V1.81

①功能

1.设备驱动文件合集

Module	Skills
platfor LED 驱动	platform_device / device tree 匹配
按键INPUT子系统	INPUT子系统
PWM 驱动 + 用户态接口	控制背光
I2C 传感器驱动	AP3216C 三合一环境光传感器
SPI 传感器驱动	ICM-20608 六轴传感器
UART	tty driver wrapper
CAN 收发框架（使用 flexcan）	netdev / socketCAN
EEPROM 驱动 + sysfs	file operations, sysfs node

用统一的CLI工具访问设备

```
myctl led on
myctl can send 01 02 03
myctl i2c read accel
```

2.加 LLM API → “语音/命令控制开发板”

②开发流程

2025.11.04 加入LED功能

1. 在设备树种创建节点

在根节点/下添加节点，描述一个LED设备

```
gpioled{
    compatible = "gpio-leds";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_led>;
```

```
    led-gpios    = <&gpio1 3 GPIO_ACTIVE_LOW>;  
    status = "okay";  
};
```

先编写一个基础的驱动框架来测试一下。

```
#include <...>  
  
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("TATAROSE");  
  
static int __init led_init(void){  
    return 0;  
}  
static void __exit led_exit(void){  
  
}  
  
module_init(led_init);  
module_exit(led_exit);
```

配置一下Cmake和Makefile,项目结构为

```
.  
├── app  
│   ├── CMakeLists.txt  
│   └── myctl.c  
├── build.sh  
├── cmake  
│   └── toolchains  
│       └── Toolchain-arm-linux-gnueabi.cmake  
├── CMakeLists.txt  
├── drivers  
│   └── LED  
│       ├── CMakeLists.txt  
│       ├── leddriver.c  
│       ├── Makefile  
│       └── modules.order  
├── Makefile  
└── scripts
```

app下存放总应用程序，**Cmake**用户态即可编译出应用程序。**drivers**下存放驱动程序，由于使用**kbuild**来编译模块，所以**Cmake**无法替代**Makefile**，所以将**Makefile**添加到**drivers/LED**。

应用程序运行结果如下：

```
/lib/modules/4.1.15/tinyLinux_IoT_kernellab/usr/bin # ./myctl
```

```
===== 设备控制程序 =====
```

```
支持的设备：
```

```
1. LED
```

```
支持的命令：
```

```
LED设备：
```

```
0 - 关闭LED
```

```
1 - 打开LED
```

```
使用方法：
```

```
交互式：myctl
```

```
命令行：myctl <device> <command>
```

```
例如：myctl led 1
```

```
=====
```

```
请选择要操作的设备：
```

```
1. LED
```

```
0. 退出
```

```
请输入选择 (0-1): 1
```

```
请选择命令：
```

```
0. 关闭
```

```
1. 打开
```

2025.11.05 添加INPUT子系统按键功能

添加按键功能，按键功能需要用到INPUT子系统，INPUT子系统是Linux内核中用于处理输入设备的子系统，如鼠标、键盘、触摸屏、游戏控制器等。添加`./drivers/INPUT_KEY/input_key.c`,修改`./CMakeLists.txt`,`./app/myctl.c`,`./app/CMakeLists.txt`。

2025.11.13 开始添加IIC传感器功能

在单片机中IIC设备通信通常需要写一个xxx.c文件，里面有些函数，如

`IIC_init()`,`IIC_read()`,`IIC_write()`，这样是把IIC控制器和IIC芯片杂糅到一起，要分离。分离出来有两部分驱动：

- IIC主机驱动：IIC控制器驱动，如`i2c_core.c`
- IIC设备驱动：IIC传感器驱动，如`i2c_sensor.c` IIC主机驱动写一次就够了，IIC设备驱动需要根据传感器的寄存器地址和数据长度进行编写。正好符合Linux驱动的分离与分层的原则。Linux内核也将IIC驱动分为两部分：
- IIC总线驱动：IIC适配器驱动。
- IIC设备驱动：IIC传感器驱动，针对具体的传感器进行编写。

IIC适配器在内核中使用`i2c_adapter`结构体，用来描述IIC总线，IIC总线驱动需要实现`i2c_adapter`结构体的成员函数。

```

struct i2c_adapter {
    struct module *owner;           // 指向拥有此适配器的模块
    unsigned int class;            // 适配器支持的I2C设备类
    const struct i2c_algorithm *algo; // 指向I2C算法结构的指针
    void *algo_data;               // 指向特定于算法的数据指针
    int timeout;                   // 超时时间（以jiffies为单位）
    int retries;                   // 重试次数
    struct device dev;             // 关联的底层设备结构体
    int nr;                        // 适配器编号（如i2c-0中的0）
    char name[48];                 // 适配器名称字符串
    struct list_head userspace_clients; // 用户空间创建的客户端链表
    const struct i2c_adapter_quirks *quirks; // 适配器特殊行为/限制描述
};

```

使用：先申请一个*i2c_adapter*结构体，然后初始化，最后将*i2c_adapter*结构体注册到内核中。
*i2c_adapter*通过*i2c_algorithm*结构体定义了底层通信方法，需要I2C适配器编写人员实现。

```

struct i2c_algorithm {
    int (*master_xfer)(struct i2c_adapter *adap, struct i2c_msg *msgs, int
num);
    int (*smbus_xfer)(struct i2c_adapter *adap, u16 addr, unsigned short
flags,
                        char read_write, u8 command, int size,
                        union i2c_smbus_data *data);
    u32 (*functionality)(struct i2c_adapter *adap);
};

```

其中*master_xfer*函数指针指向具体实现I2C消息传输的函数，这是I2C通信的核心。I.MX6ULL的I2C控制器驱动时*i2c_imx.c*。

```

struct i2c_client {
    unsigned short flags;           // 标志位，如I2C_CLIENT_TEN表示使用10位地址
    unsigned short addr;           // 芯片地址（注意：只存储7位地址，位于低7位）
    char name[I2C_NAME_SIZE];      // 设备类型名称
    struct i2c_adapter *adapter;   // 所属的I2C适配器
    struct i2c_driver *driver;     // 设备的驱动程序
    struct device dev;             // 设备模型中的设备结构
    int irq;                       // 设备产生的IRQ（如果有的话）
    struct list_head detected;     // 用于链接到i2c_driver.clients列表
#ifdef IS_ENABLED(CONFIG_I2C_SLAVE)
    i2c_slave_cb_t slave_cb;      // 从模式的回调函数
#endif
};

```

*i2c_client*结构体描述I2C设备，I2C设备驱动需要实现*i2c_client*结构体的成员函数。内核会在启动时或检测到设备时自动创建*i2c_client*结构体，驱动开发者不需要手动定义。我们需要在设备树中添加具体的i2c设备节点，并指定设备类型。比如：

```
&i2c1 {
    clock-frequency = <100000>;
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_i2c1>;
    status = "okay";

    mag3110@0e {
        compatible = "fsl,mag3110";
        reg = <0x0e>;
        position = <2>;
    };

    fxls8471@1e {
        compatible = "fsl,fxls8471";
        reg = <0x1e>;
        position = <0>;
        interrupt-parent = <&gpio5>;
        interrupts = <0 8>;
    };
};
```

这样系统在解析设备树时，就会自动创建*i2c_client*结构体，并注册到内核中。

*i2c_driver*结构体描述I2C设备驱动,找个需要开发者创建.



- 驱动开发者实现*i2c_driver*结构体并注册到I2C子系统
- 内核根据设备树或ACPI信息自动创建*i2c_client*结构体
- I2C核心将*i2c_driver*与匹配的*i2c_client*绑定
- 调用*probe*函数初始化设备
- 设备可以正常工作，通过*i2c_client*与硬件通信
- 当设备移除或驱动卸载时，调用*remove*函数清理资源

在使用设备树的时候，I2C设备挂载到指定节点下,修改设备树：

- 设备树io初始化

```
pinctrl-names = "default"; //i2c1引脚复用
pinctrl-0 = <&pinctrl_i2c1>; //i2c1引脚复用
```

设置pinctrl_i2c1:

```
pinctrl_i2c1: i2c1grp {
    fsl,pins = <
        MX6UL_PAD_UART4_TX_DATA__I2C1_SCL 0x4001b8b0
        MX6UL_PAD_UART4_RX_DATA__I2C1_SDA 0x4001b8b0
    >;
};
```

其中MX6UL_PAD_UART4_TX_DATA__I2C1_SCL要去看原理图上的连接关系，再去imx6ul-pinctrl.h中找对应的宏定义。注意在同一个iic节点下，多个设备不能有相同的地址。修改好之后重新生成设备树文件。编写驱动框架，I2C字符设备驱动框架。初始化AP3216C，实现ap3216c_read()函数。接下来通过i2c_transfer()函数进行数据传输。

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num);
```

struct i2c_adapter *adap: 表示要使用的I2C适配器对象 struct i2c_msg *msgs: 指向i2c_msg结构体数组的指针，每个结构体代表一次单独的消息传递操作 int num: 数组中包含的消息数量

- 成功时返回传输成功的消息数量
- 发生错误时返回负数表示失败原因 i2c_transfer函数本身并不直接驱动硬件完成消息交互，而是通过以下流程工作：

寻找到对应的i2c_adapter;找到该适配器关联的i2c_algorithm;调用i2c_algorithm中的master_xfer()函数;master_xfer()函数才是真正驱动硬件完成实际消息传输的接口

2025.11.18 添加SPI六轴传感器

主机控制器驱动：SOC的SPI外设驱动，此驱动是半导体原厂编写好的，当spi控制器和驱动匹配时，probe函数会调用，完成驱动初始化。

SPI控制器驱动的核心就是spi_master构建，注册和注销。